

Lecture 03: Loops in C++ & Pattern Printing

Loops programming ka sabse powerful tool hain — ye ek hi code block ko baar baar execute karne ki ability dete hain bina repetition likhe. Is lecture mein hum for, while, aur do-while loops ko detail mein samjhenge, saath mein nested loops se pattern printing seekhenge. Ye concepts competitive programming aur real-world coding ke liye foundation hain.



Introduction to Loops

What are Loops

Loop ek control structure hai jo code ke ek block ko repeatedly execute karta hai jab tak ek specific condition true hai. Real life mein socho — agar aapko ek room ke har tile ko saf karna hai, to aap ek tile saf karoge, phir next, phir next — ye process tab tak repeat hogi jab tak saari tiles saf na ho jaye. Programming mein loop exactly yehi kaam karta hai.

INTUITION

Loop = Automated Repetition. Bina loop ke 100 baar cout likhna padega. Loop se ek baar likho, aur wo 100 baar execute hoga. Ye **DRY Principle** (Don't Repeat Yourself) ka core example hai.

Types of Loops in C++

Loop Type	Best For	Condition Check	Minimum Runs
for loop	Known iteration count	Before execution	0 times
while loop	Unknown iteration count	Before execution	0 times
do-while loop	At least once run chahiye	After execution	1 time



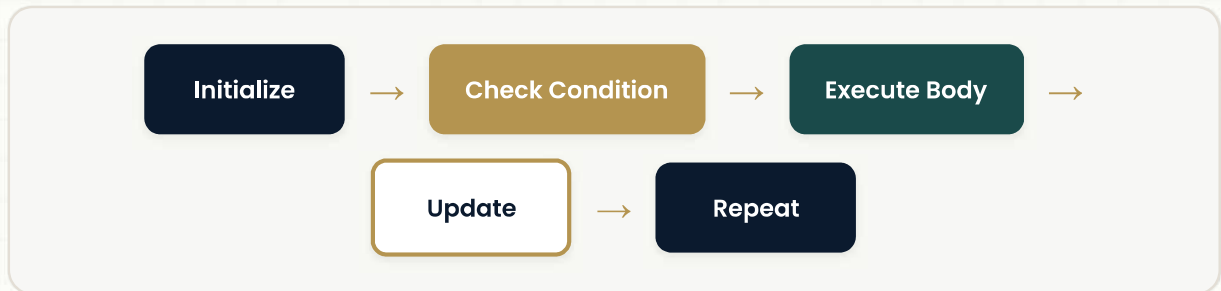
For Loop

Syntax and Usage

for loop tab use hota hai jab aapko pata ho ki kitni baar repeat karna hai. Isme teen parts hote hain: **initialization** , **condition** , aur **update** .

```
for(initialization; condition; update) {  
    // code to repeat  
}
```

How It Works Internally



Execution flow:

1. **Step 1:** Initialization — loop variable set hota hai (sirf ek baar)
2. **Step 2:** Condition check — agar true to body execute, false to exit
3. **Step 3:** Body execute — loop ka actual code chalta hai
4. **Step 4:** Update — loop variable modify hota hai
5. **Step 5:** Step 2 par wapas jaate hain — repeat until condition false

Example: Print 1 to 5

```
for(int i = 1; i ≤ 5; i++) {  
    cout << i << endl;  
}
```

Step-by-Step Execution

```
Step 1: i = 1 (initialization)
Step 2: Check i ≤ 5? true → Print 1
Step 3: i++ → i becomes 2
Step 4: Check i ≤ 5? true → Print 2
Step 5: i++ → i becomes 3
Step 6: Check i ≤ 5? true → Print 3
Step 7: i++ → i becomes 4
Step 8: Check i ≤ 5? true → Print 4
Step 9: i++ → i becomes 5
Step 10: Check i ≤ 5? true → Print 5
Step 11: i++ → i becomes 6
Step 12: Check i ≤ 5? false → EXIT LOOP
```

Key Insight: for loop mein initialization *sirf ek baar* hota hai, condition *har iteration se pehle* check hoti hai, aur update *har iteration ke baad* hota hai. Ye order hamesha same rehta hai.

Example: Print Even Numbers

```
for(int i = 2; i ≤ 10; i += 2) {
    cout << i << " ";
}
// Output: 2 4 6 8 10
```

Isme update step `i += 2` hai, isliye har iteration mein 2 se badhta hai. Isse even numbers print hote hain.

Example: Sum of First 10 Numbers

```
int sum = 0;

for(int i = 1; i ≤ 10; i++) {
    sum = sum + i; // sum += i; (same thing)
}

cout << "Sum: " << sum << endl;
// Output: Sum: 55
```

INTUITION

Accumulator Pattern: sum variable ko accumulator kehte hain. Har iteration mein ek value add hoti hai. Isi pattern se factorial, product, average, aur saari aggregate calculations hoti hain. Hamesha accumulator ko `0` se initialize karo (multiplication ke case mein `1` se).

Example: Multiplication Table

```
int num = 5;

cout << "Multiplication table of " << num << endl;

for(int i = 1; i ≤ 10; i++) {
    cout << num << " x " << i << " = " << (num * i) << endl;
}
```

Example: Reverse Counting

```
for(int i = 10; i ≥ 1; i--) {
    cout << i << " ";
}
// Output: 10 9 8 7 6 5 4 3 2 1
```

EDGE CASE

Infinite Loop Trap: Agar update step galat ho to loop kabhi end nahi hoga. `for(int i = 1; i <= 5; i--)` // *WRONG: i-- se i kam hota, condition hamesha true* `for(int i = 1; i >= 5; i++)` // *WRONG: i 1 se 5 tak kabhi nahi pahunchega* // *Correct: for(int i=1; i<=5; i++)*



While Loop

Syntax and Usage

while loop tab use hota hai jab aapko nahi pata ki exactly kitni baar repeat karna hai. Condition check pehle hoti hai, isliye agar condition initially false hai to loop ek baar bhi nahi chalega.

```
while(condition) {
    // code to repeat
}
```

Example: Print 1 to 5

```
int i = 1;

while(i ≤ 5) {
    cout << i << endl;
    i++;
}
```

Example: Password Validation

```
int password;
int correctPassword = 1234;

cout << "Enter password: ";
cin >> password;

while(password != correctPassword) {
    cout << "Wrong! Try again: ";
    cin >> password;
}

cout << "Access granted!" << endl;
```

INTUITION

while = "Jab tak". while loop ko Hindi mein "jab tak" socho: "Jab tak password galat hai, tab tak pucho." Ye user input validation, file reading, aur network polling jaise scenarios mein perfect fit hai jahan iteration count pehle se unknown hai.

Example: Sum Until User Enters 0

```
int num;
int sum = 0;

cout << "Enter numbers (0 to stop):" << endl;

while(true) {
    cin >> num;

    if(num == 0) {
        break; // Exit loop immediately
    }

    sum += num;
}

cout << "Total sum: " << sum << endl;
```



Do-While Loop

Syntax and Usage

do-while loop **kam se kam ek baar** execute hota hai, kyunki condition check **body ke baad** hoti hai. Ye tab useful hai jab aapko pehli iteration guaranteed chahiye ho.

```
do {  
    // code to repeat  
} while(condition);
```

Example: Print 1 to 5

```
int i = 1;  
  
do {  
    cout << i << endl;  
    i++;  
} while(i ≤ 5);
```

While vs Do-While Comparison

```
// while loop - condition pehle check hoti hai  
int i = 10;  
  
while(i ≤ 5) { // Condition false, loop kabhi nahi chalega  
    cout << i << endl;  
}  
// Output: (nothing)  
  
// do-while loop - condition baad mein check hoti hai  
int i = 10;  
  
do {  
    cout << i << endl; // Ek baar to chalega hi  
} while(i ≤ 5);  
// Output: 10
```

Key Insight: do-while loop ka sabse common use case **menu-driven programs** hain. User se choice pucho, action perform karo, phir pucho ki continue karna hai ya nahi. Pehli baar to menu dikhana hi hai, isliye do-while perfect hai.



Loop Control Statements

Break Statement

break loop ko immediately exit kar deta hai. Jaise hi break execute hota hai, control loop ke baad wale statement par chala jata hai.

```
for(int i = 1; i ≤ 10; i++) {
    if(i == 5) {
        break; // Stop when i is 5
    }
    cout << i << " ";
}
// Output: 1 2 3 4
```

IMPORTANT

break sirf **innermost loop** ko exit karta hai. Nested loops mein outer loop continue rehta hai. Agar saare nested loops exit karne hain to flags use karo ya goto (avoid karo), ya better — function mein wrap karo aur return use karo.

Continue Statement

continue current iteration ko skip kar deta hai aur control sidha update step par bhej deta hai. Body ka baaki code execute nahi hota.

```
for(int i = 1; i ≤ 5; i++) {
    if(i == 3) {
        continue; // Skip 3
    }
    cout << i << " ";
}
// Output: 1 2 4 5
```

Example: Skip Even Numbers

```
for(int i = 1; i ≤ 10; i++) {
    if(i % 2 == 0) {
        continue; // Skip even numbers
    }
    cout << i << " ";
}
// Output: 1 3 5 7 9
```

INTUITION

break = "Bas, ab nahi" (loop khatam). **continue** = "Ye wala skip karo, next dekho" (iteration skip). break loop se bahar nikalta hai, continue same loop mein next iteration par jump karta hai.



Pattern Printing with Nested Loops

The Core Concept

Pattern printing **nested loops** par based hota hai. Outer loop **rows** control karta hai, inner loop **columns** control karta hai. Har pattern ka logic row aur column ke relationship par depend karta hai.

Key Insight: Har pattern ko solve karne ka mantra: *"Outer loop = rows, Inner loop = columns"*. Pehle outer loop ki range fix karo, phir inner loop mein socho ki har row mein kitne columns chahiye. Ye relationship row number se derive hota hai.

Pattern 1: Square Pattern

```
*****
*****
*****
*****
*****
```

```
for(int i = 1; i ≤ 5; i++) {           // Outer: 5 rows
    for(int j = 1; j ≤ 5; j++) {       // Inner: 5 columns per row
        cout << "*";
    }
    cout << endl;                     // New line after each row
}
```

Pattern 2: Right Triangle (Increasing)

```
*
**
***
****
*****
```

```
for(int i = 1; i ≤ 5; i++) {           // Row 1 to 5
    for(int j = 1; j ≤ i; j++) {       // Columns = row number
        cout << "*";
    }
    cout << endl;
}
```

INTUITION

Row-Column Relationship: Row 1 mein 1 star, Row 2 mein 2 stars, Row 3 mein 3 stars... Isliye inner loop **j ≤ i** hai. Jitni row number, utne stars. Ye "increasing triangle" ka fundamental logic hai.

Pattern 3: Right Triangle (Decreasing)

```
*****
****
***
**
*
```

```
for(int i = 5; i ≥ 1; i--) {           // Row 5 to 1
    for(int j = 1; j ≤ i; j++) {       // Columns = current row number
        cout << "*";
    }
    cout << endl;
}
```

Pattern 4: Inverted Right Triangle (With Spaces)

```

*
**
***
****
*****
```

```
for(int i = 1; i ≤ 5; i++) {
    // Print spaces
    for(int j = 1; j ≤ 5-i; j++) {
        cout << " ";
    }
    // Print stars
    for(int j = 1; j ≤ i; j++) {
        cout << "*";
    }
    cout << endl;
}
```

WORKFLOW

Space-Star Pattern Logic:

1. Total width fix karo (yahan 5)
2. Spaces = Total Width - Current Row Number
3. Stars = Current Row Number
4. Formula: **spaces = n - i** , **stars = i**

Row 1: 4 spaces + 1 star

Row 2: 3 spaces + 2 stars

Row 3: 2 spaces + 3 stars

Row 4: 1 space + 4 stars

Row 5: 0 spaces + 5 stars

Pattern 5: Number Triangle

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
for(int i = 1; i ≤ 5; i++) {
    for(int j = 1; j ≤ i; j++) {
        cout << j << " ";
    }
    cout << endl;
}
```

Pattern 6: Pyramid (Full Triangle)

```
  *
 ***
*****
*****
*****
```

```
int n = 5;

for(int i = 1; i ≤ n; i++) {
    // Spaces: n - i
    for(int j = 1; j ≤ n - i; j++) {
        cout << " ";
    }
    // Stars: 2*i - 1
    for(int j = 1; j ≤ 2*i - 1; j++) {
        cout << "*";
    }
    cout << endl;
}
```

INTUITION

Pyramid Pattern Formula: Row **i** mein stars ki count **2i - 1** hoti hai. Kyunki:

Row 1: 1 star ($2*1-1 = 1$)

Row 2: 3 stars ($2*2-1 = 3$)

Row 3: 5 stars ($2*3-1 = 5$)

Ye odd number sequence hai. Spaces har row mein center karne ke liye hain: **n - i**.



Common Mistakes and Gotchas

EDGE CASE

- 1. Off-by-One Error:** `for(int i = 1; i < 5; i++)` // Prints 1,2,3,4 – 5 nahi print hoga! `for(int i = 1; i <= 5; i++)` // Correct: prints 1,2,3,4,5
- 2. Missing Update:** `int i = 1; while(i <= 5) { cout << i; }` // INFINITE LOOP – i kabhi badhta nahi!
- 3. Semicolon after loop:** `for(int i = 1; i <= 5; i++); { cout << i; }` // Semicolon makes empty loop body!
- 4. Variable scope:** `for(int i = 0; i < 5; i++) { } cout << i;` // ERROR: i is out of scope



Loop Comparison Summary

Feature	for	while	do-while
Initialization	Inside loop header	Before loop	Before loop
Condition Check	Before each iteration	Before each iteration	After each iteration
Update	Inside loop header	Inside body	Inside body
Minimum Executions	0	0	1
Best Use Case	Known iterations	Unknown iterations	Menu/input validation
Code Readability	Compact	Flexible	Guaranteed first run



Key Takeaways

TAKEAWAY

- **for loop** use karo jab iteration count pehle se pata ho — initialization, condition, aur update ek line mein
- **while loop** use karo jab condition-driven repetition ho aur count unknown ho
- **do-while loop** use karo jab kam se kam ek baar execution guaranteed chahiye ho
- **break** loop ko immediately exit karta hai; **continue** current iteration skip karke next par jump karta hai
- Nested loops mein **outer loop = rows** , **inner loop = columns** . Har pattern ka logic row-column relationship se derive hota hai
- Space-star patterns mein formula: **spaces = n - i** , **stars = 2i - 1** (pyramid ke liye)
- Infinite loops se bachne ke liye hamesha ensure karo ki condition eventually false ho jaye
- Accumulator pattern (sum, product) mein variable ko correct initial value do: 0 for sum, 1 for product

INTERVIEW TIP

Commonly asked loop questions:

1. **for vs while vs do-while:** for = known count, while = unknown count, do-while = at least once. Condition check ka timing main difference hai.
2. **break vs continue:** break loop exit karta hai, continue iteration skip karta hai. break nested loops mein sirf innermost loop exit karta hai.
3. **Time Complexity:** Single loop = $O(n)$, Nested loop = $O(n^2)$, Triple nested = $O(n^3)$. Pattern printing mostly $O(n^2)$.
4. **Infinite loop causes:** Missing update, wrong condition, semicolon after for().
5. **Pattern printing approach:** Pehle row count fix karo, phir har row mein spaces aur stars ka formula derive karo. Dry run karo pen-paper se.